Chapter 24

- **Project Management Concepts**

Slide Set to accompany

Software Engineering: A Practitioner's Approach, 7/e

by Roger S. Pressman

The Four P's

- People — People must be organized to perform s/w work
- Product — Communicating with stake holders for requirements
- Process — Select appropriate process for people and product
- Project — Project must be planned by estimating effort and calendar time to accomplish work tasks like:
  - Defining work
  - Establishing checkpoints
  - Identifying mechanisms for controlling and monitoring.

Effective s/w Project Management focuses on the Four P's

- People — the most important element of a successful project
- Product — the software to be built
- Process — the set of framework activities and software engineering tasks to get the job done
- Project — all work required to make the product a reality

- Managers who forgets SE is an human endeavour – never have success in project mgmt
- Manager who fails stakeholder communication – risks building solution irrelevant to problem
- Manager who pays little attention to process – risks of misusing technical methods and tools
- Manager who embarks without solid PM – jeopardizes the success of project

The management spectrum

- People
- Goal is cultivation of motivated, highly skilled s/w people i.e. people factor is very important
- Thus SE institute developed a people capability Maturity model (People-CMM) that specifies that every organization needs to continually improve its ability to attract, develop, motivate, organize and retain the workforce to accomplish business objectives
- People –CMM is a companion to S/w-CMM in creating mature product

- Product

Before planning for project :

1. Product Objectives and scope should be established
2. Alternative solutions should be identified
3. Technical constraints and management constraints should be identified

- Objectives identify the overall goals for the product (from stakeholders point of view)
- Scope identifies the primary data, functions and behaviors that characterized the product
  - If these two things are not done: it is impossible to define estimates of cost, effective assessment of risk, a realistic breakdown of project tasks, or a manageable project schedule that provides indication of progress

- Process
- A software process provides the framework fro which a comprehensive plan for s/w development can be established
- A number of task sets: tasks, milestones, work products and quality assurance points enable framework activities to be adapted for project
- Finally umbrella activities such as: quality assurance, s/w configuration management and measurement overlay the process model
- These umbrella activities are independent of any framework activity and occur throughout the process

- Project

- The primary reason for conducting planned and controlled s/w projects as it is the only known way to manage complexity and still s/w teams struggle.
- A study of 250 large s/w projects between 1998 and 2004 shows:

- ✔ 25 were deemed successful in that they achieved their schedule, cost and quality objectives
- ✔ 5o had delays or overruns below 35 percent
- ✔ 175 experienced major delays and overruns or were terminated without completion

To avoid project failure

- S/w project manager and s/w engineers who build the product must avoid a set of common warning signs, understand the critical success factors that lead to good project management and develop a commonsense approach for planning, monitoring and controlling the project

People

- In a study three vice-presidents of major technology companies were asked what was the most important contributor to a successful s/w project
  - VP1 : If one thing has to be picked then I'd say it is not the tools we use, it's the people
  - VP2: most important ingredient for success of project was having smart people and the organizations ability to recruit good people
  - VP3: only rule I have in management is to ensure I have good people…. Real good people and I grow good people and provide an environment in which good people can produce
- The next section discusses the stakeholders who participate in the software process and the manner in which they are organized to perform effective SE

Stakeholders can be categorized into one of five constituencies

1. *Senior managers* who define the business issues that often have significant influence on the project.
2. *Project (technical) managers* who must plan, motivate, organize, and control the practitioners who do software work.
3. *Practitioners* who deliver the technical skills that are necessary to engineer a product or application.
4. *Customers* who specify the requirements for the software to be engineered and other stakeholders who have a peripheral interest in the outcome.
5. *End-users* who interact with the software once it is released for production use.

Every project is populated by people who fall within this taxonomy. To be effective the project team must be organized in a way that maximizes each persons skills and abilities – This is the job of team leader

Software Teams

How to lead?

How to organize?

How to collaborate?

How to motivate?

How to create good ideas?

Team Leaders

Team Leader

- The MOI Model
  - **Motivation.** The ability to encourage (by "push or pull") technical people to produce to their best ability.
  - **Organization.** The ability to mold existing processes (or invent new ones) that will enable the initial concept to be translated into a final product.
  - **Ideas or innovation.** The ability to encourage people to create and feel creative even when they must work within bounds established for a particular software product or application.

Software Teams

The following factors must be considered when selecting a software project team structure ...

- the difficulty of the problem to be solved
- the size of the resultant program(s) in lines of code or function points
- the time that the team will stay together (team lifetime)
- the degree to which the problem can be modularized
- the required quality and reliability of the system to be built
- the rigidity of the delivery date
- the degree of sociability (communication) required for the project

Organizational Paradigms

- closed paradigm—structures a team along a traditional hierarchy of authority
- random paradigm—structures a team loosely and depends on individual initiative of the team members
- open paradigm—attempts to structure a team in a manner that achieves some of the controls associated with the closed paradigm but also much of the innovation that occurs when using the random paradigm
- synchronous paradigm—relies on the natural compartmentalization of a problem and organizes team members to work on pieces of the problem with little active communication among themselves

suggested by Constantine [Con93]

Avoid Team "Toxicity"

- A frenzied work atmosphere in which team members waste energy and lose focus on the objectives of the work to be performed.
- High frustration caused by personal, business, or technological factors that cause friction among team members.
- "Fragmented or poorly coordinated procedures" or a poorly defined or improperly chosen process model that becomes a roadblock to accomplishment.
- Unclear definition of roles resulting in a lack of accountability and resultant finger-pointing.
- "Continuous and repeated exposure to failure" that leads to a loss of confidence and a lowering of morale.

Agile Teams

- Team members must have trust in one another.
- The distribution of skills must be appropriate to the problem.
- Mavericks may have to be excluded from the team, if team cohesiveness is to be maintained.
- Team is “self-organizing”
  - An adaptive team structure
  - Uses elements of Constantine’s random, open, and synchronous paradigms
  - Significant autonomy

Team Coordination & Communication

- *Formal, impersonal approaches* include software engineering documents and work products (including source code), technical memos, project milestones, schedules, and project control tools (Chapter 23), change requests and related documentation, error tracking reports, and repository data (see Chapter 26).
- *Formal, interpersonal procedures* focus on quality assurance activities (Chapter 25) applied to software engineering work products. These include status review meetings and design and code inspections.
- *Informal, interpersonal procedures* include group meetings for information dissemination and problem solving and "collocation of requirements and development staff."
- *Electronic communication* encompasses electronic mail, electronic bulletin boards, and by extension, video-based conferencing systems.
- *Interpersonal networking* includes informal discussions with team members and those outside the project who may have experience or insight that can assist team members.

The Product Scope

- Scope
  - **Context.** How does the software to be built fit into a larger system, product, or business context and what constraints are imposed as a result of the context?
  - **Information objectives.** What customer-visible data objects (Chapter 8) are produced as output from the software? What data objects are required for input?
  - **Function and performance.** What function does the software perform to transform input data into output? Are any special performance characteristics to be addressed?
- Software project scope must be unambiguous and understandable at the management and technical levels.

Problem Decomposition

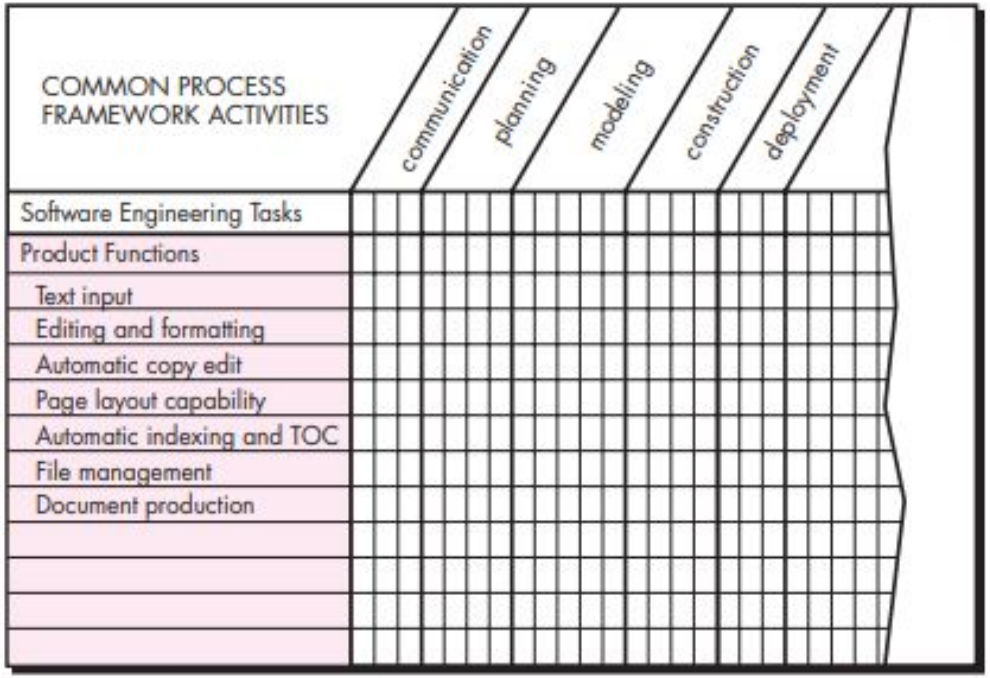
- Sometimes called *partitioning* or *problem elaboration*
- Once scope is defined …
  - It is decomposed into constituent functions
  - It is decomposed into user-visible data objects

  or
  - It is decomposed into a set of problem classes
- Decomposition process continues until all functions or problem classes have been defined

The Process

- Once a process framework has been established
  - Consider project characteristics
  - Determine the degree of rigor required
  - Define a task set for each software engineering activity
    - Task set =
      - Software engineering tasks
      - Work products
      - Quality assurance points
      - Milestones

Melding the Problem and the Process



The Project

- *Projects get into trouble when …*
  - Software people don't understand their customer's needs.
  - The product scope is poorly defined.
  - Changes are managed poorly.
  - The chosen technology changes.
  - Business needs change [or are ill-defined].
  - Deadlines are unrealistic.
  - Users are resistant.
  - Sponsorship is lost [or was never properly obtained].
  - The project team lacks people with appropriate skills.
  - Managers [and practitioners] avoid best practices and lessons learned.

Common-Sense Approach to Projects

- *Start on the right foot.* This is accomplished by working hard (very hard) to understand the problem that is to be solved and then setting realistic objectives and expectations.
- *Maintain momentum. The* project manager must provide incentives to keep turnover of personnel to an absolute minimum, the team should emphasize quality in every task it performs, and senior management should do everything possible to stay out of the team's way.
- *Track progress.* For a software project, progress is tracked as work products (e.g., models, source code, sets of test cases) are produced and approved (using formal technical reviews) as part of a quality assurance activity.
- *Make smart decisions.* In essence, the decisions of the project manager and the software team should be to "keep it simple."
- *Conduct a postmortem analysis.* Establish a consistent mechanism for extracting lessons learned for each project.

To Get to the Essence of a Project

- Why is the system being developed?
- What will be done?
- When will it be accomplished?
- Who is responsible?
- Where are they organizationally located?
- How will the job be done technically and managerially?
- How much of each resource (e.g., people, software, tools, database) will be needed?

Barry Boehm [Boe96]

Critical Practices

- Formal risk management
- Empirical cost and schedule estimation
- Metrics-based project management
- Earned value tracking
- Defect tracking against quality targets
- People aware project management

Chapter 27

- **Project Scheduling**

Slide Set to accompany

Software Engineering: A Practitioner's Approach, 7/e

by Roger S. Pressman

Why Are Projects Late?

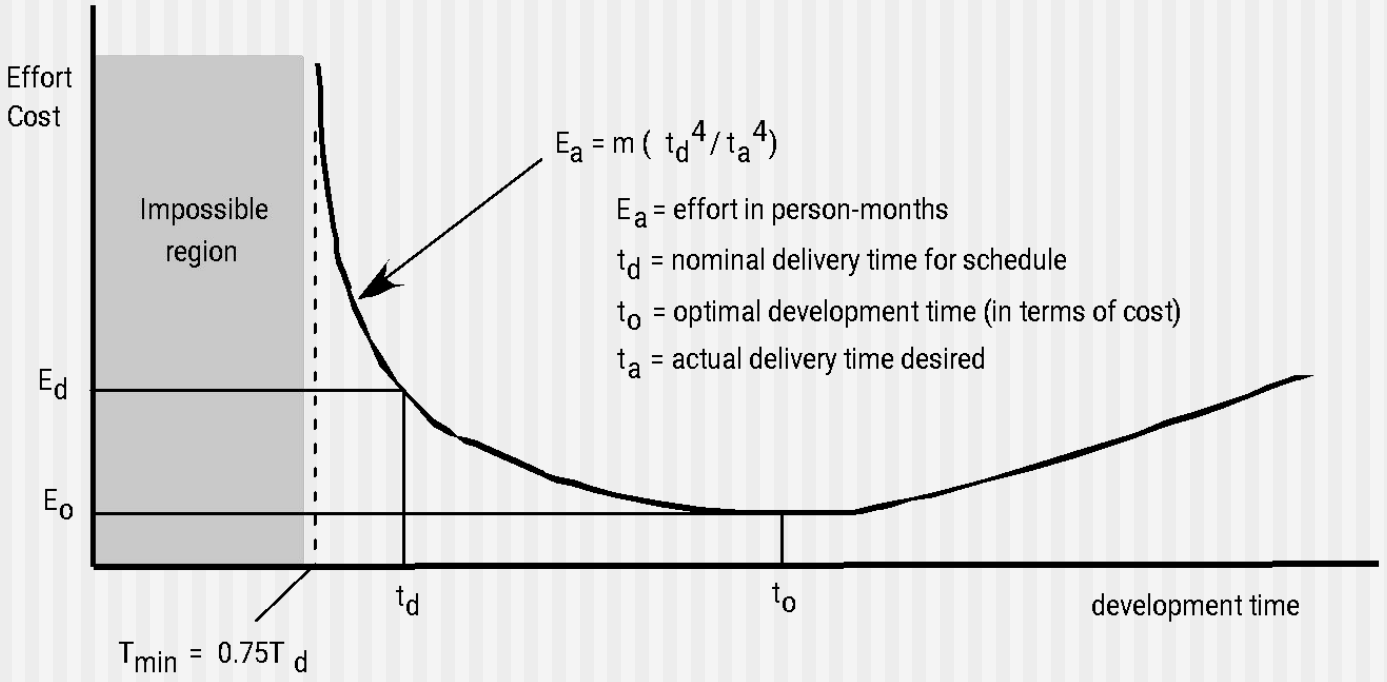
- an unrealistic deadline established by someone outside the software development group
- changing customer requirements that are not reflected in schedule changes;
- an honest underestimate of the amount of effort and/or the number of resources that will be required to do the job;
- predictable and/or unpredictable risks that were not considered when the project commenced;
- technical difficulties that could not have been foreseen in advance;
- human difficulties that could not have been foreseen in advance;
- miscommunication among project staff that results in delays;
- a failure by project management to recognize that the project is falling behind schedule and a lack of action to correct the problem

- So, what to do?
- To recommend the following steps in this situation:
- 1. Perform a detailed estimate using historical data from past projects. Determine the estimated effort and duration for the project.
- 2. Using an incremental process model (Chapter 2), develop a software engineering strategy that will deliver critical functionality by the imposed deadline, but delay other functionality until later. Document the plan.
- 3. Meet with the customer and (using the detailed estimate), explain why the imposed deadline is unrealistic. Be certain to note that all estimates are based on performance on past projects.
- 4. Offer the incremental development strategy as an alternative

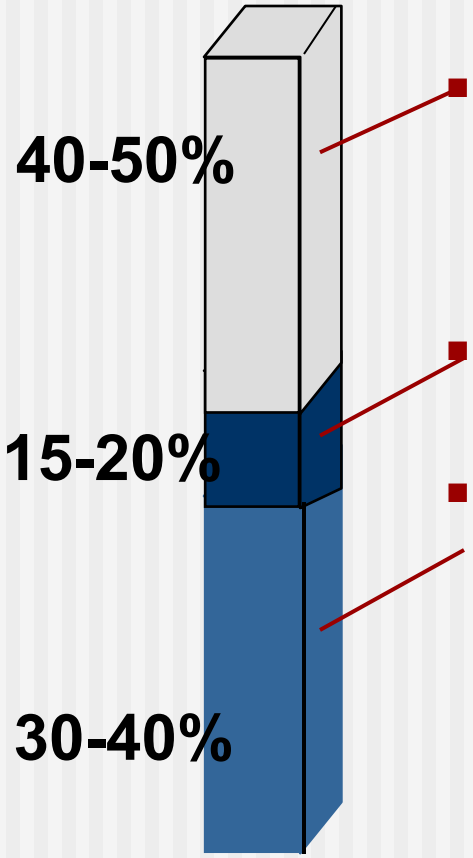
Scheduling Principles

- compartmentalization—define distinct tasks
- interdependency—indicate task interrelationship
- effort validation—be sure resources are available
- defined responsibilities—people must be assigned
- defined outcomes—each task must have an output
- defined milestones—review for quality

Effort and Delivery Time



Effort Allocation



- "front end" activities
  - customer communication
  - analysis
  - design
  - review and modification
- construction activities
  - coding or code generation
- testing and installation
  - unit, integration
  - white-box, black box
  - regression

Defining Task Sets

- determine type of project
- assess the degree of rigor required
- identify adaptation criteria
- select appropriate software engineering tasks

- Comprehensive taxonomy of software project types, most software
- organizations encounter the following projects:
- 1. Concept development projects that are initiated to explore some new business concept or application of some new technology.
- 2. New application development projects that are undertaken as a consequence of a specific customer request.
- 3. Application enhancement projects that occur when existing software undergoes major modifications to function, performance, or interfaces that are observable by the end user.
- 4. Application maintenance projects that correct, adapt, or extend existing software in ways that may not be immediately obvious to the end user.
- 5. Reengineering projects that are undertaken with the intent of rebuilding an existing (legacy) system in whole or in part.

1.1 **Concept scoping** determines the overall scope of the project.

1.2 **Preliminary concept planning** establishes the organization's ability to undertake the work implied by the project scope.

1.3 **Technology risk assessment** evaluates the risk associated with the technology to be implemented as part of the project scope.

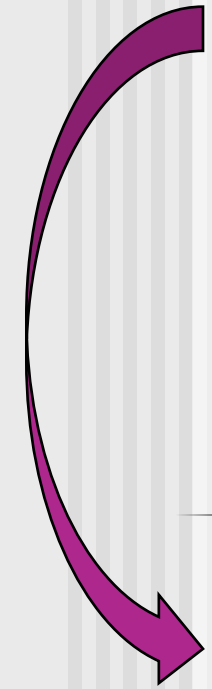
1.4 **Proof of concept** demonstrates the viability of a new technology in the software context.

1.5 **Concept implementation** implements the concept representation in a manner that can be reviewed by a customer and is used for "marketing" purposes when a concept must be sold to other customers or management.

1.6 **Customer reaction** to the concept solicits feedback on a new technology concept and targets specific customer applications.

Task Set Refinement

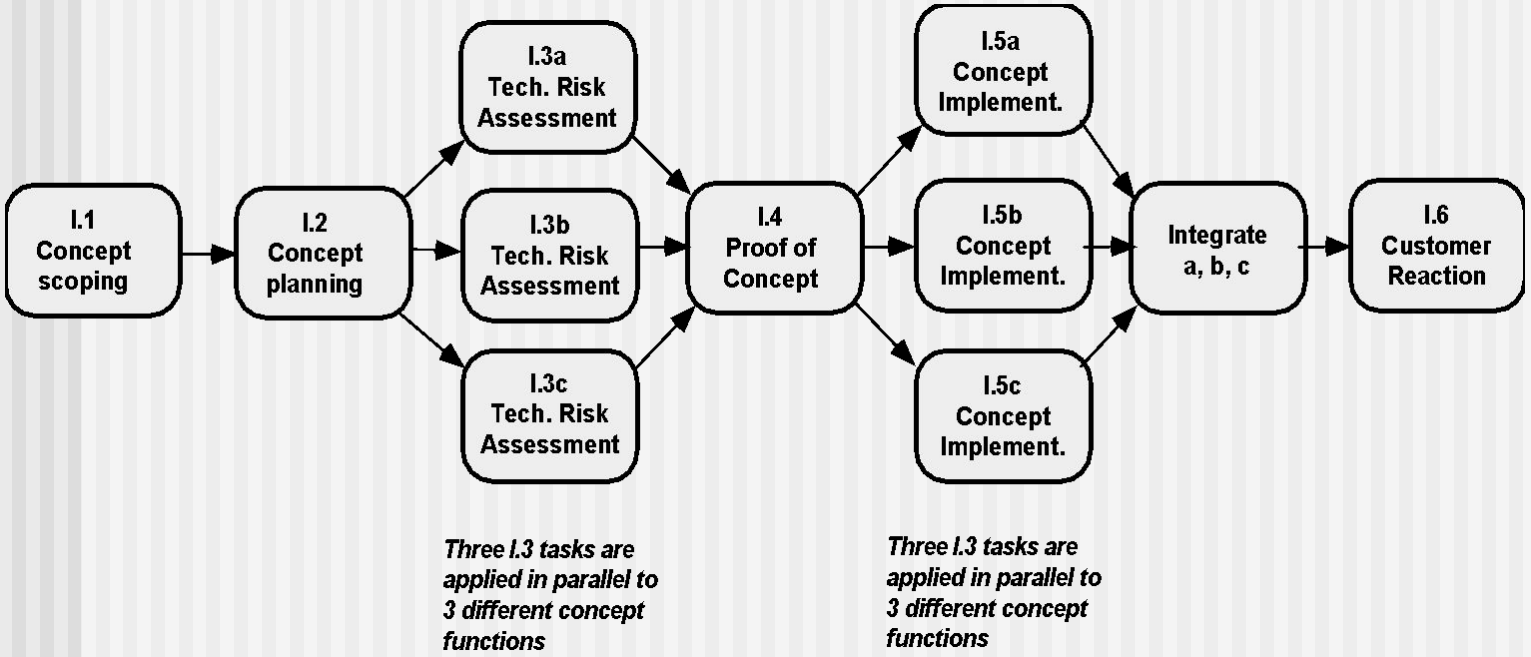
1.1 **Concept scoping** determines the overall scope of the project.



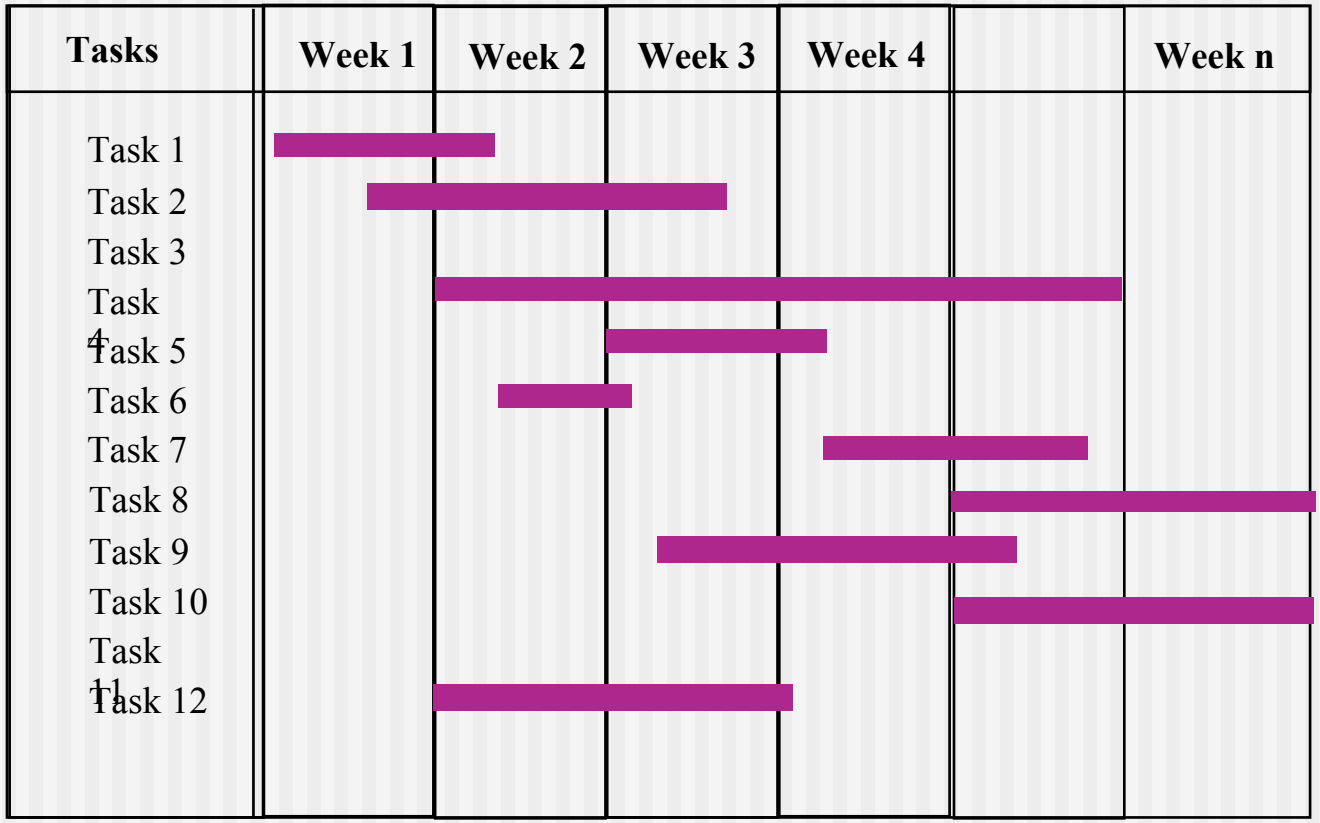
is refined to

Task definition: Task 1.1 Concept Scoping
1.1.1 Identify need, benefits and potential customers;
1.1.2 Define desired output/control and input events that drive the application;
Begin Task 1.1.2
1.1.2.1 FTR: Review written description of need
FTR indicates that a formal technical review (Chapter 26) is to be conducted.
1.1.2.2 Derive a list of customer visible outputs/inputs
1.1.2.3 FTR: Review outputs/inputs with customer and revise as required;
endtask Task 1.1.2
1.1.3 Define the functionality/behavior for each major function;
Begin Task 1.1.3
1.1.3.1 FTR: Review output and input data objects derived in task 1.1.2;
1.1.3.2 Derive a model of functions/behaviors;
1.1.3.3 FTR: Review functions/behaviors with customer and revise as required;
endtask Task 1.1.3
1.1.4 Isolate those elements of the technology to be implemented in software;
1.1.5 Research availability of existing software;
1.1.6 Define technical feasibility;
1.1.7 Make quick estimate of size;
1.1.8 Create a Scope Definition;
endTask definition: Task 1.1

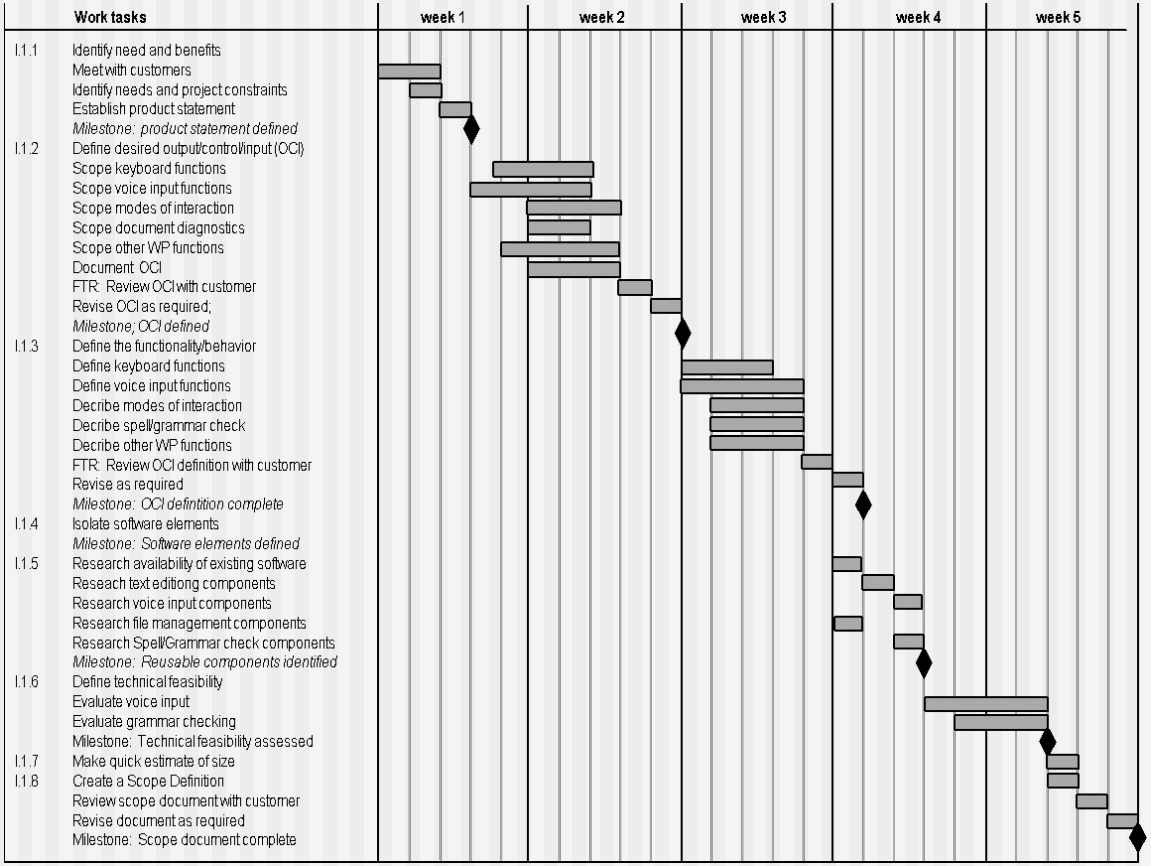
Define a Task Network



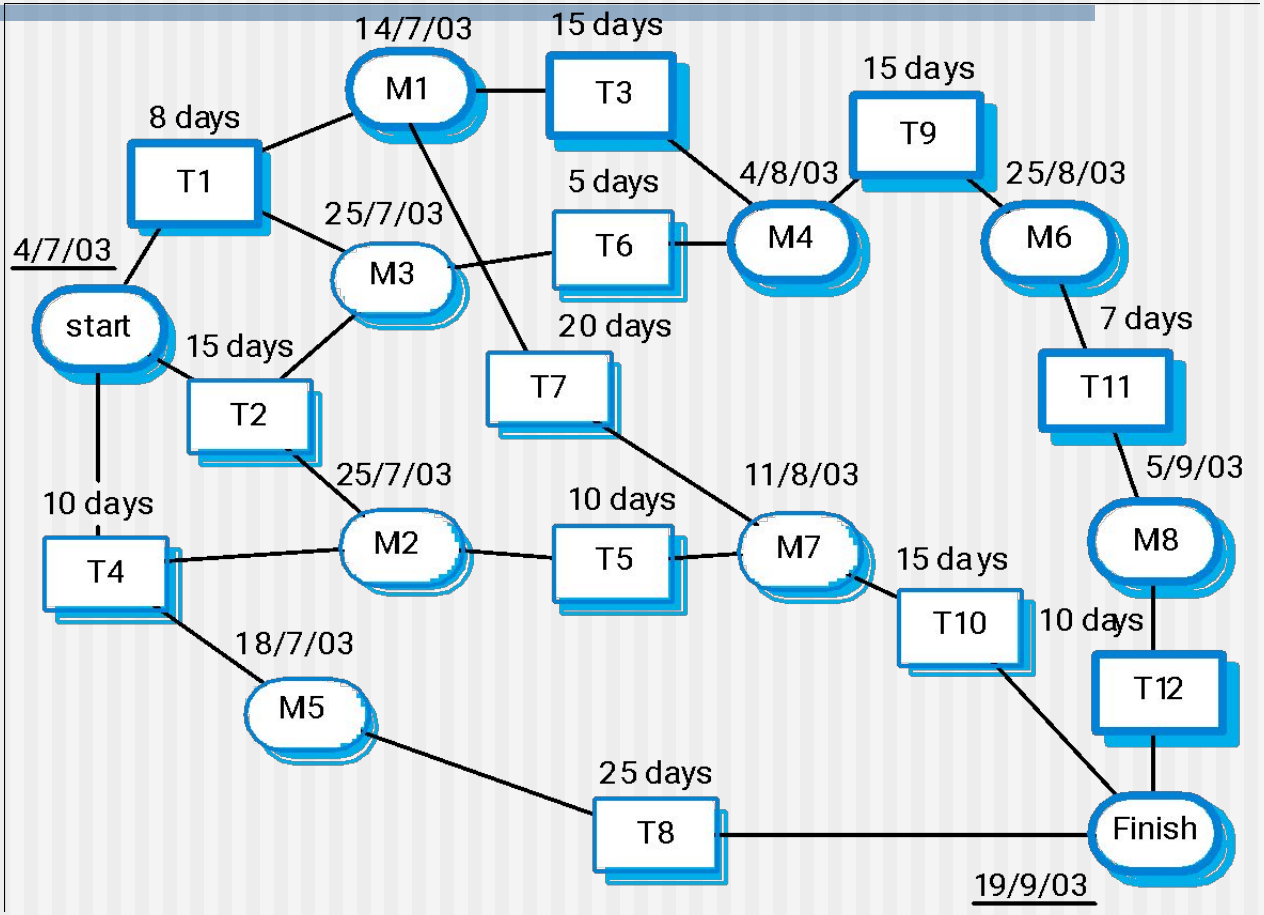
Timeline Charts

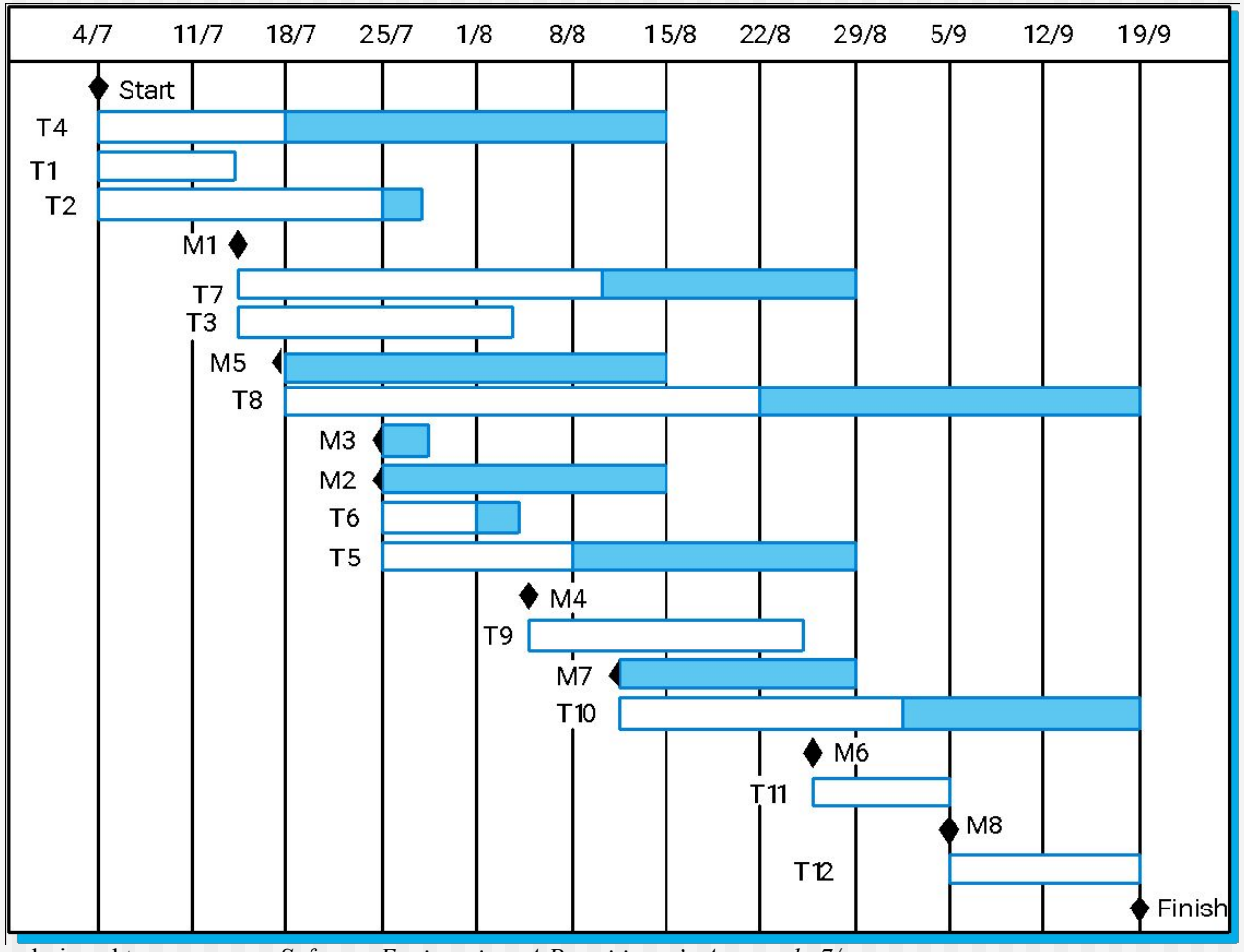


Use Automated Tools to Derive a Timeline Chart



Activity	Duration (days)	Dependencies
T1	8	
T2	15	
T3	15	T1 (M1)
T4	10	
T5	10	T2, T4 (M2)
T6	5	T1, T2 (M3)
T7	20	T1 (M1)
T8	25	T4 (M5)
T9	15	T3, T6 (M4)
T10	15	T5, T7 (M7)
T11	7	T9 (M6)
T12	10	T11 (M8)


14/7/03
15 days
M1
T3
15 days
T9
8 days
T1
4/8/03
25/8/03
5 days
25/7/03
T6
M4
M6
4/7/03
M3
start
20 days
7 days
15 days
T7
T11
T2
25/7/03
11/8/03
5/9/03
10 days
10 days
M2
M8
T4
T5
M7
15 days
T10
10 days
18/7/03
T12
M5
25 days
T8
Finish
19/9/03



Schedule Tracking

- conduct periodic project status meetings in which each team member reports progress and problems.
- evaluate the results of all reviews conducted throughout the software engineering process.
- determine whether formal project milestones (the diamonds shown in Figure 27.3) have been accomplished by the scheduled date.
- compare actual start-date to planned start-date for each project task listed in the resource table (Figure 27.4).
- meet informally with practitioners to obtain their subjective assessment of progress to date and problems on the horizon.
- use earned value analysis (Section 27.6) to assess progress quantitatively.

Progress on an OO Project-I

- *Technical milestone: OO analysis completed*
  - All classes and the class hierarchy have been defined and reviewed.
  - Class attributes and operations associated with a class have been defined and reviewed.
  - Class relationships (Chapter 8) have been established and reviewed.
  - A behavioral model (Chapter 8) has been created and reviewed.
  - Reusable classes have been noted.
- *Technical milestone: OO design completed*
  - The set of subsystems (Chapter 9) has been defined and reviewed.
  - Classes are allocated to subsystems and reviewed.
  - Task allocation has been established and reviewed.
  - Responsibilities and collaborations (Chapter 9) have been identified.
  - Attributes and operations have been designed and reviewed.
  - The communication model has been created and reviewed.

Progress on an OO Project-II

- *Technical milestone: OO programming completed*
  - Each new class has been implemented in code from the design model.
  - Extracted classes (from a reuse library) have been implemented.
  - Prototype or increment has been built.
- *Technical milestone: OO testing*
  - The correctness and completeness of OO analysis and design models has been reviewed.
  - A class-responsibility-collaboration network (Chapter 6) has been developed and reviewed.
  - Test cases are designed and class-level tests (Chapter 19) have been conducted for each class.
  - Test cases are designed and cluster testing (Chapter 19) is completed and the classes are integrated.
  - System level tests have been completed.

Earned Value Analysis (EVA)

- Earned value
  - is a measure of progress
  - enables us to assess the "percent of completeness" of a project using quantitative analysis rather than rely on a gut feeling
  - "provides accurate and reliable readings of performance from as early as 15 percent into the project." [Fle98]

Computing Earned Value-I

- The *budgeted cost of work scheduled* (BCWS) is determined for each work task represented in the schedule.
  - $BCWS_i$ is the effort planned for work task *i*.
  - To determine progress at a given point along the project schedule, the value of BCWS is the sum of the $BCWS_i$ values for all work tasks that should have been completed by that point in time on the project schedule.
- The BCWS values for all work tasks are summed to derive the *budget at completion,* BAC. Hence,

$$BAC = \sum (BCWS_k) \text{ for all tasks } k$$

Computing Earned Value-II

- Next, the value for *budgeted cost of work performed* (BCWP) is computed.
  - The value for BCWP is the sum of the BCWS values for all work tasks that have actually been completed by a point in time on the project schedule.
- "the distinction between the BCWS and the BCWP is that the former represents the budget of the activities that were planned to be completed and the latter represents the budget of the activities that actually were completed." [Wil99]
- Given values for BCWS, BAC, and BCWP, important progress indicators can be computed:
  - Schedule performance index, SPI = BCWP/BCWS
  - Schedule variance, SV = BCWP – BCWS
  - SPI is an indication of the efficiency with which the project is utilizing scheduled resources.

Computing Earned Value-III

- Percent scheduled for completion = BCWS/BAC
  - provides an indication of the percentage of work that should have been completed by time *t*.
- Percent complete = BCWP/BAC
  - provides a quantitative indication of the percent of completeness of the project at a given point in time, *t*.
- *Actual cost of work performed,* ACWP, is the sum of the effort actually expended on work tasks that have been completed by a point in time on the project schedule. It is then possible to compute
  - Cost performance index, CPI = BCWP/ACWP
  - Cost variance, CV = BCWP – ACWP

Chapter 28

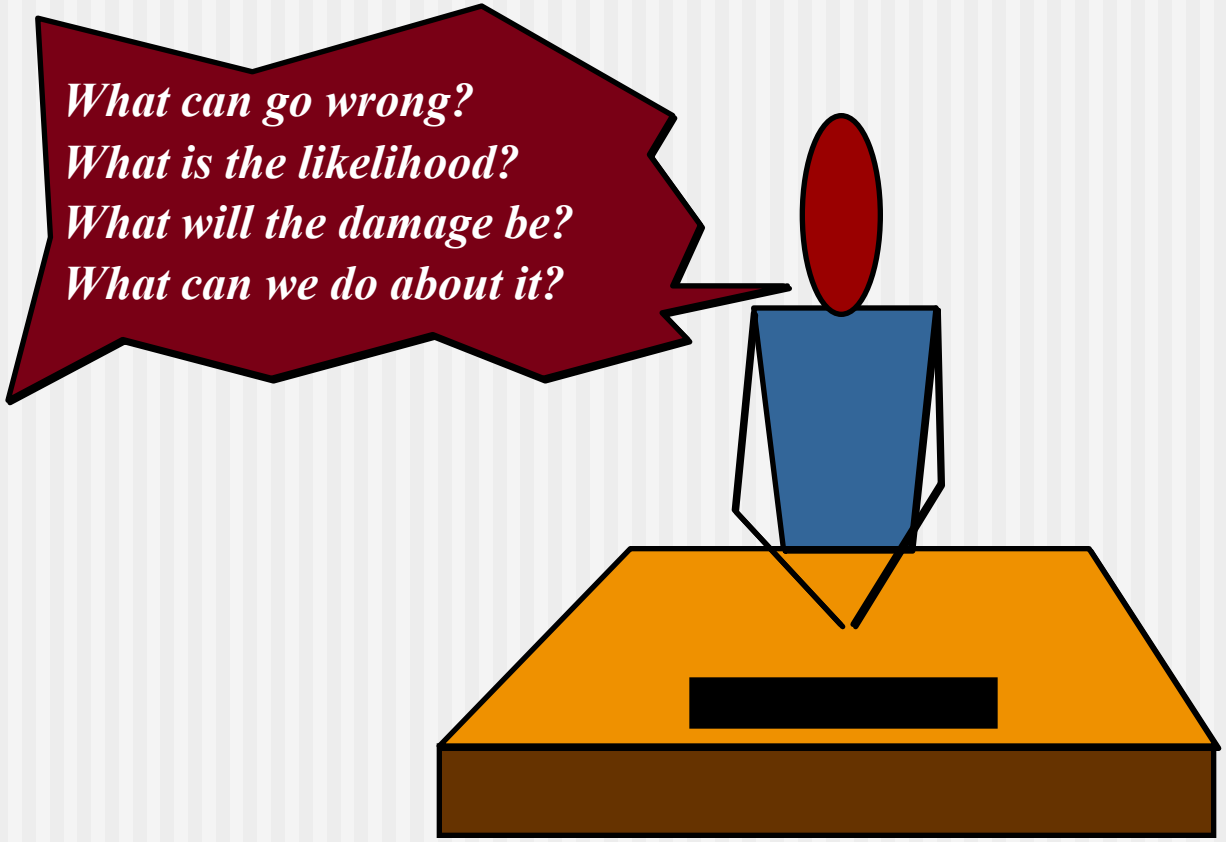
- **Risk Analysis**

Slide Set to accompany

Software Engineering: A Practitioner's Approach, 7/e

by Roger S. Pressman

Project Risks



- Peter Drucker [Dru75] once said, “While it is futile to try to eliminate risk, and questionable to try to minimize it, it is essential that the risks taken be the right risks.”

- Before you can identify the “right risks” to be taken during a software project, it is important to identify all risks that are obvious to both managers and practitioners.

Reactive Risk Management

- At best, a reactive strategy monitors the project for likely risks.
- Resources are set aside to deal with them, should they become actual problems.
- More commonly, the software team does nothing about risks until something goes wrong.
- Then, the team flies into action in an attempt to correct the problem rapidly. This is often called a fire-fighting mode.
- When this fails, "crisis management" [Cha92] takes over and the project is in real jeopardy.

Reactive Risk Management

- project team reacts to risks when they occur
- mitigation—plan for additional resources in anticipation of fire fighting
- fix on failure—resource are found and applied when the risk strikes
- crisis management—failure does not respond to applied resources and project is in jeopardy

Proactive Risk Management

- A considerably more intelligent strategy for risk management is to be proactive.
- A proactive strategy begins long before technical work is initiated.
- Potential risks are identified, their probability and impact are assessed, and they are ranked by importance.
- Then, the software team establishes a plan for managing risk.
- The primary objective is to avoid risk, but because not all risks can be avoided, the team works to develop a contingency plan that will enable it to respond in a controlled and effective manner.

Proactive Risk Management

- formal risk analysis is performed
- organization corrects the root causes of risk
  - TQM concepts and statistical SQA
  - examining risk sources that lie beyond the bounds of the software
  - developing the skill to manage change

Software risks

- Although there has been considerable debate about the proper definition for software risk, there is general agreement that risk always involves two characteristics:

1. Uncertainty—the risk may or may not happen; that is, there are no 100 probable risks and
2. Loss—if the risk becomes a reality, unwanted consequences or losses will occur [Hig95].

- When risks are analyzed, it is important to quantify the level of uncertainty and the degree of loss associated with each risk.
- To accomplish this, different categories of risks are considered.

- Project risks threaten the project plan. That is, if project risks become real, it is likely that the project schedule will slip and that costs will increase.
- Project risks identify potential budgetary, schedule, personnel (staffing and organization), resource, stakeholder, and requirements problems and their impact on a software project.
- The project complexity, size, and the degree of structural uncertainty were also defined as project (and estimation) risk factors.

- Technical risks threaten the quality and timeliness of the software to be produced. If a technical risk becomes a reality, implementation may become difficult or impossible.
- Technical risks identify potential design, implementation, interface, verification, and maintenance problems.
- In addition, specification ambiguity, technical uncertainty, technical obsolescence, and "leading-edge" technology are also risk-factors.
- Technical risks occur because the problem is harder to solve than you thought it would be.

- Business risks threaten the viability of the software to be built and often jeopardize the project or the product.
- Candidates for the top five business risks are:

(1) building an excellent product or system that no one really wants (market risk),

(2) building a product that no longer fits into the overall business strategy for the company(strategic risk),

(3) building a product that the sales force doesn't understand how to sell (sales risk),

(4) losing the support of senior management due to a change in focus or a change in people (management risk), and

(5) losing budgetary or personnel commitment (budget risks).

- Another general categorization of risks has been proposed by Charette [Cha89].
- Known risks are those that can be uncovered after careful evaluation of the project plan, the business and technical environment in which the project is being developed, and other reliable information sources (e.g., unrealistic delivery date, lack of documented requirements or software scope, poor development environment).
- Predictable risks are extrapolated from past project experience (e.g., staff turnover, poor communication with the customer, dilution of staff effort as ongoing maintenance requests are serviced).
- Unpredictable risks are the joker in the deck. They can and do occur, but they are extremely difficult to identify in advance.

Seven Principles of Risk Management

- **Maintain a global perspective**—view software risks within the context of system and the business problem
- **Take a forward-looking view**—think about the risks that may arise in the future; establish contingency plans
- **Encourage open communication**—if someone states a potential risk, don't discount it.
- **Integrate**—a consideration of risk must be integrated into the software process
- **Emphasize a continuous process**—the team must be vigilant throughout the software process, modifying identified risks as more information is known and adding new ones as better insight is achieved.
- **Develop a shared product vision**—if all stakeholders share the same vision of the software, it likely that better risk identification and assessment will occur.
- **Encourage teamwork**—the talents, skills and knowledge of all stakeholder should be pooled

Risk Management Paradigm

Risk Identification

- Risk identification is a systematic attempt to specify threats to the project plan (estimates, schedule, resource loading, etc.). By identifying known and predictable risks, and controlling them when necessary.
- There are two distinct types of risks for each of the generic risks and product-specific risks.

1. Generic risks are a potential threat to every software project.
2. Product-specific risks can be identified only by those with a clear understanding of the technology, the people, and the environment that is specific to the software that is to be built.

- To identify product-specific risks, the project plan and the software statement of scope are examined, and an answer to the following question is developed: "What special characteristics of this product may threaten our project plan?"
- One method for identifying risks is to create a risk item checklist.
- The checklist can be used for risk identification and focuses on some subset of known and predictable risks in the following generic subcategories:

Risk Identification

- *Product size*—risks associated with the overall size of the software to be built or modified.
- *Business impact*—risks associated with constraints imposed by management or the marketplace.
- *Customer characteristics*—risks associated with the sophistication of the customer and the developer's ability to communicate with the customer in a timely manner.
- *Process definition*—risks associated with the degree to which the software process has been defined and is followed by the development organization.
- *Development environment*—risks associated with the availability and quality of the tools to be used to build the product.
- *Technology to be built*—risks associated with the complexity of the system to be built and the "newness" of the technology that is packaged by the system.
- *Staff size and experience*—risks associated with the overall technical and project experience of the software engineers who will do the work.

Assessing Project Risk-I

- Have top software and customer managers formally committed to support the project?
- Are end-users enthusiastically committed to the project and the system/product to be built?
- Are requirements fully understood by the software engineering team and their customers?
- Have customers been involved fully in the definition of requirements?
- Do end-users have realistic expectations?

Assessing Project Risk-II

- Is project scope stable?
- Does the software engineering team have the right mix of skills?
- Are project requirements stable?
- Does the project team have experience with the technology to be implemented?
- Is the number of people on the project team adequate to do the job?
- Do all customer/user constituencies agree on the importance of the project and on the requirements for the system/product to be built?

Risk Components and drivers

- The U.S. Air Force [AFC88] has published a pamphlet that contains excellent guidelines for software risk identification and abatement.
- The Air Force approach requires that the project manager identify the risk drivers that affect software risk components—
- performance, cost, support, and schedule.
- The risk components are defined in the following manner:

Risk Components

- *performance risk*—the degree of uncertainty that the product will meet its requirements and be fit for its intended use.
- *cost risk*—the degree of uncertainty that the project budget will be maintained.
- *support risk*—the degree of uncertainty that the resultant software will be easy to correct, adapt, and enhance.
- *schedule risk*—the degree of uncertainty that the project schedule will be maintained and that the product will be delivered on time.

- The impact of each risk driver on the risk component is divided into one of four impact categories—negligible, marginal, critical, or catastrophic

Components / Category		Performance	Support	Cost	Schedule
Catastrophic	1	Failure to meet the requirement would result in mission failure		Failure results in increased costs and schedule delays with expected values in excess of $500K	
	2	Significant degradation to nonachievement of technical performance	Nonresponsive or unsupportable software	Significant financial shortages, budget overrun likely	Unachievable IOC
Critical	1	Failure to meet the requirement would degrade system performance to a point where mission success is questionable		Failure results in operational delays and/or increased costs with expected value of $100K to $500K	
	2	Some reduction in technical performance	Minor delays in software modifications	Some shortage of financial resources, possible overruns	Possible slippage in IOC
Marginal	1	Failure to meet the requirement would result in degradation of secondary mission		Costs, impacts, and/or recoverable schedule slips with expected value of $1K to $100K	
	2	Minimal to small reduction in technical performance	Responsive software support	Sufficient financial resources	Realistic, achievable schedule
Negligible	1	Failure to meet the requirement would create inconvenience or nonoperational impact		Error results in minor cost and/or schedule impact with expected value of less than $1K	
	2	No reduction in technical performance	Easily supportable software	Possible budget underrun	Early achievable IOC

Risk Projection

- *Risk projection*, also called *risk estimation,* attempts to rate each risk in two ways
  - the likelihood or probability that the risk is real
  - the consequences of the problems associated with the risk, should it occur.
- The are four risk projection steps:
  - establish a scale that reflects the perceived likelihood of a risk
  - delineate the consequences of the risk
  - estimate the impact of the risk on the project and the product,
  - note the overall accuracy of the risk projection so that there will be no misunderstandings.

Building a Risk Table

Risks	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End-users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet expectations	TE	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	
Σ				
Σ				
Σ				

Impact values:
1—catastrophic
2—critical
3—marginal
4—negligible

Building the Risk Table

- Estimate the probability of occurrence
- Estimate the impact on the project on a scale of 1 to 5, where
  - 1 = low impact on project success
  - 5 = catastrophic impact on project success
- sort the table by probability and impact

Risk Exposure (Impact)

The overall *risk exposure,* RE, is determined using the following relationship [Hal98]:

RE = *P* x *C*

where

P is the probability of occurrence for a risk, and

C is the cost to the project should the risk occur.

Risk Exposure Example

- **Risk identification.** Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.
- **Risk probability.** 80% (likely).
- **Risk impact.** 60 reusable software components were planned. If only 70 percent can be used, 18 components would have to be developed from scratch (in addition to other custom software that has been scheduled for development). Since the average component is 100 LOC and local data indicate that the software engineering cost for each LOC is $14.00, the overall cost (impact) to develop the components would be 18 x 100 x 14 = $25,200.
- **Risk exposure.** RE = 0.80 x 25,200 ~ $20,200.

Risk Mitigation, Monitoring, and Management

- mitigation—how can we avoid the risk?
- monitoring—what factors can we track that will enable us to determine if the risk is becoming more or less likely?
- management—what contingency plans do we have if the risk becomes a reality?

Risk Due to Product Size

Attributes that affect risk:

- **estimated size of the product in LOC or FP?**
- **estimated size of product in number of programs, files, transactions?**
- **percentage deviation in size of product from average for previous products?**
- **size of database created or used by the product?**
- **number of users of the product?**
- **number of projected changes to the requirements for the product? before delivery? after delivery?**
- **amount of reused software?**

Risk Due to Business Impact

Attributes that affect risk:

- **affect of this product on company revenue?**
- **visibility of this product by senior management?**
- **reasonableness of delivery deadline?**
- **number of customers who will use this product**
- **interoperability constraints**
- **sophistication of end users?**
- **amount and quality of product documentation that must be produced and delivered to the customer?**
- **governmental constraints**
- **costs associated with late delivery?**
- **costs associated with a defective product?**

Risks Due to the Customer

Questions that must be answered:

- **Have you worked with the customer in the past?**
- **Does the customer have a solid idea of requirements?**
- **Has the customer agreed to spend time with you?**
- **Is the customer willing to participate in reviews?**
- **Is the customer technically sophisticated?**
- **Is the customer willing to let your people do their job—that is, will the customer resist looking over your shoulder during technically detailed work?**
- **Does the customer understand the software engineering process?**

Risks Due to Process Maturity

Questions that must be answered:

- **Have you established a common process framework?**
- **Is it followed by project teams?**
- **Do you have management support for software engineering**
- **Do you have a proactive approach to SQA?**
- **Do you conduct formal technical reviews?**
- **Are CASE tools used for analysis, design and testing?**
- **Are the tools integrated with one another?**
- **Have document formats been established?**

Technology Risks

Questions that must be answered:

- **Is the technology new to your organization?**
- **Are new algorithms, I/O technology required?**
- **Is new or unproven hardware involved?**
- **Does the application interface with new software?**
- **Is a specialized user interface required?**
- **Is the application radically different?**
- **Are you using new software engineering methods?**
- **Are you using unconventional software development methods, such as formal methods, AI-based approaches, artificial neural networks?**
- **Are there significant performance constraints?**
- **Is there doubt the functionality requested is "do-able?"**

Staff/People Risks

Questions that must be answered:

- **Are the best people available?**
- **Does staff have the right skills?**
- **Are enough people available?**
- **Are staff committed for entire duration?**
- **Will some people work part time?**
- **Do staff have the right expectations?**
- **Have staff received necessary training?**
- **Will turnover among staff be low?**

Recording Risk Information

Project: Embedded software for XYZ system
Risk type: schedule risk
Priority (1 low ... 5 critical): 4
Risk factor: Project completion will depend on tests which require hardware component under development. Hardware component delivery may be delayed
Probability: 60 %
Impact: Project completion will be delayed for each day that hardware is unavailable for use in software testing
Monitoring approach:
Scheduled milestone reviews with hardware group
Contingency plan:
Modification of testing strategy to accommodate delay using software simulation
Estimated resources: 6 additional person months beginning in July

RMMM Plan

- A risk management strategy can be included in the software project plan, or the risk management steps can be organized into a separate *risk mitigation, monitoring, and management plan* (RMMM).
- The RMMM plan documents all work performed as part of risk analysis and is used by the project manager as part of the overall project plan.

- Risk monitoring is a project tracking activity with three primary objectives:
- (1) to assess whether predicted risks do, in fact, occur; (2) to ensure that risk aversion steps defined for the risk are being properly applied; and

  (3) to collect information that can be used for future risk analysis.
- In many cases, the problems that occur during a project can be traced to more than one risk.
- Another job of risk monitoring is to attempt to allocate origin [what risk(s) caused which problems throughout the project].

Risk Information sheet

Risk ID: P02-4-32	Date: 5/9/09	Prob: 80%	Impact: high

Description:
Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.

Refinement/context:
Subcondition 1: Certain reusable components were developed by a third party with no knowledge of internal design standards.
Subcondition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components.
Subcondition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.

Mitigation/monitoring:
1. Contact third party to determine conformance with design standards.
2. Press for interface standards completion; consider component structure when deciding on interface protocol.
3. Check to determine number of components in subcondition 3 category; check to determine if language support can be acquired.

Management/contingency plan/trigger:
RE computed to be $20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly.
Trigger: Mitigation steps unproductive as of 7/1/09.

Current status:
5/12/09: Mitigation steps initiated.

Originator: D. Gagne	Assigned: B. Laster